

claude-windows / c7679509-ee2b-4443-b240-0b3b1b1a7291

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c7679509-ee2b-4443-b240-0b3b1b1a7291\subagents\workflows\wf_a2b97cff-dcd\agent-ae787e5663e508304.jsonl`
- SHA-256: `43c2cd04db87216f8b06da274d8a19654c39d87b076e851a6fa73af15fc8a7f3`
- Source modified: `2026-06-21T14:00:53+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_a2b97cff-dcd`
- session_id: `c7679509-ee2b-4443-b240-0b3b1b1a7291`
```

Transcript

1. user (2026-06-21T13:56:09.275Z)

Adversarially VERIFY the load-bearing claims below about packaging/distributing our Python app. For EACH claim, try to REFUTE it (check feasibility, licensing accuracy, version facts, whether it actually applies to a torch+opencv+ffmpeg+FastAPI app). Default to "uncertain" if you cannot confirm. Use web search to check current facts.

TOPIC: launcher-ux-wrapper

CLAIMS:

```
[
  {
    "claim": "D10 locks delivery as 'a self-hosted webserver + UI the user spins up (NOT a desktop app)... the user points at local storage + GPU/CPU OR cloud locations and the UI drives the tool', and explicitly SUPERSEDES DESKTOP_APP_PLAN.md.",
    "confidence": "high",
    "source": "C:/Users/avidu/Projects/khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md:39"
  },
  {
    "claim": "The engine today starts via a console entry main() with a --server flag (python -m backend.main --server --port 8000); there is NO browser auto-open, no system tray, and no launcher in main.py ? grep for webbrowser/tray/--open returns only def main() and the --server arg.",
    "confidence": "high",
    "source": "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/main.py:556,562"
  },
  {
    "claim": "The engine server is hard-bound to 127.0.0.1:8000, so the browser-on-localhost model is the structural reality the launch UX must wrap.",
    "confidence": "high",
    "source": "C:/Users/avidu/Projects/badminton-highlight-indexer/docs/.../LOCAL_APPLIANCE_ARCHITECTURE.md:14,30 (citing backend/main.py:638)"
  },
  {
    "claim": "GET /api/health exists and returns {status, sport, default_segmenter}, so a launcher can poll readiness before opening the browser.",
    "confidence": "high",
    "source": "C:/Users/avidu/Projects/khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md:47 (citing backend/main.py:127)"
  },
  {
    "claim": "The hard packaging problem is real and load-bearing: native WASB runs in a SEPARATE conda env (torch 1.11+cu113) while the main engine uses torch 2.6+cu124; ffmpeg/ffprobe are system deps; WASB weights live in a GCS bucket ? so a single frozen binary cannot represent the runtime.",
    "confidence": "high",
    "source": "orchestrator-provided KEY FACTS + DESKTOP_APP_PLAN.md:37 (de-WSL native runner) and LOCAL_APPLIANCE_ARCHITECTURE.md:96 (torch-1.11 conda env makes the image non-trivial)"
  }
]
```

```

},
{
  "claim": "PyInstaller-bundling PyTorch with CUDA produces 3-5GB executables and CUDA/cuDNN
shared libs 'cannot be split'; small frozen torch apps are CPU-only/older-CUDA ? making
PyInstaller a poor fit for our GPU app.",
  "confidence": "high",
  "source": "https://github.com/orgs/pyinstaller/discussions/8552 and
https://discuss.pytorch.org/t/.../194822"
},
{
  "claim": "Transformer Lab (Electron + Python/PyTorch API server) explicitly REJECTED
PyInstaller because users install plugins that add arbitrary Python libs, so they needed a
modifiable post-install Python env; they ship an isolated Miniconda installed to an app-specific
dir with pinned deps and defensive conda handling.",
  "confidence": "high",
  "source": "https://lab.cloud/blog/packaging%20python/"
},
{
  "claim": "The Tauri+FastAPI sidecar pattern is proven, but its most common runtime failure
is silent backend-start failure when port 8000 is already in use; startup is 5-10s with model
load; Windows UTF-8 crashes are a known gotcha.",
  "confidence": "high",
  "source": "https://aiechoes.substack.com/p/building-production-ready-desktop"
},
{
  "claim": "Tauri on Windows requires the WebView2 runtime: preinstalled on Windows 11 (and
recent Win10), but on older Windows the Tauri installer must download/install it, or bundle a
fixed runtime (+~180MB).",
  "confidence": "high",
  "source": "https://v2.tauri.app/distribute/windows-installer/ and
https://v2.tauri.app/reference/webview-versions/"
},
{
  "claim": "pystray supports Windows/macOS/Linux with full feature parity for icon + menu +
Quit, but is licensed LGPL-3.0 ? not in the project's MIT/Apache/BSD code guardrail, so dynamic-
link compatibility needs explicit sign-off or an MIT substitute.",
  "confidence": "high",
  "source": "https://github.com/moses-palmer/pystray (license) +
https://pystray.readthedocs.io/en/latest/usage.html (platform support)"
},
{
  "claim": "Inno Setup is an open-source Windows installer builder supporting single-EXE
distribution, non-admin installs, and Pascal scripting (usable to fetch GCS weights / set up the
env); winget can ingest INNO/EXE/MSI installer types.",
  "confidence": "high",
  "source": "https://jrsoftware.org/isinfo.php + https://learn.microsoft.com/en-
us/windows/package-manager/winget/"
},
{
  "claim": "Mature local-AI apps (Open WebUI/Ollama) treat first-run as: open localhost in a
browser, create an account, then enter provider URL + API key in settings, and pull models from
an in-app browser ? a precedent for an in-UI (not OS-installer) first-run wizard for the Gemini
key / compute choice.",
  "confidence": "medium",
  "source": "https://docs.openwebui.com/getting-started/quick-start/connect-a-provider/"
},
{
  "claim": "Compute placement is already a setting surfaced from GET /api/capabilities (truly-
local / cloud-serving), so the 'pick GPU-or-cloud' wizard step maps onto an existing engine seam
rather than net-new config.",
  "confidence": "high",
  "source": "C:/Users/avidu/Projects/khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md:35,90-100
(D6 + compute matrix); engine backend/config/presets.py PROFILE_PRESETS"
}
]

```

RECOMMENDATION UNDER TEST: Adopt (b)+(d): a tiny system-tray LAUNCHER that owns the server lifecycle and auto-opens the browser, distributed by a one-click Inno Setup installer. This SATISFIES D10's intent rather than violating it ? the rendered product stays the khelsutra SPA in the user's own browser at 127.0.0.1:8000 (no embedded webview, no UI fork); the tray is only a friendly on/off switch + status light + language toggle, i.e. exactly 'a self-hosted webserver the user spins up' made non-scary. A launcher is NOT a desktop app in D10's sense; option (c) (Electron/Tauri embedded-webview shell) IS, and it should be rejected: it contradicts D10's letter (the superseded DESKTOP_APP_PLAN), adds a WebView2/Chromium runtime burden on varied Indian Windows hardware, and collides head-on with the hard packaging problem because its sidecar pattern presumes a PyInstaller freeze that blows up to 3-5GB with CUDA and can't host our separate WASB conda env.\n\nSequencing (smallest-first, evidence-graduated, matching repo norms): Slice 1 ? ship option (a) immediately: have the `indexer` console entry optionally health-poll then webbrowser.open after --server (a few stdlib lines, zero new deps, zero license risk) so the auto-open habit lands first. Slice 2 ? wrap it in the (b) pystray tray (Open/Restart/View logs/Language/Quit, free-port fallback for the documented port-8000-busy failure, crash balloon, graceful child-process kill), FIRST resolving the pystray LGPL-3.0 guardrail question (get a dynamic-link OK or swap to an MIT tray lib). Slice 3 ? the (d) Inno Setup installer that provisions a managed env the Transformer-Lab way (isolated Miniconda/uv in an app-specific dir for the dual-torch + WASB env, locate/ship LGPL ffmpeg, fetch WASB weights from GCS on first run with a progress UI), drops a Start-Menu shortcut to the tray launcher, and (optionally) a winget manifest. Budget a code-signing cert so SmartScreen/AV don't scare non-tech users off the .exe.\n\nFirst-run wizard: put it IN the web UI (the Open WebUI/Ollama precedent), launched on the tray app's first open, with 3 steps grounded in existing seams ? (1) Gemini key: paste-or-skip, stored locally, with the consent copy that ?1280px chunks leave the machine (never a training source per the guardrail); (2) Compute: pick Local GPU / Local CPU / Cloud, rendered from GET /api/capabilities so it only offers what the box can honestly do (truly-local vs cloud-serving presets) and disables options the hardware can't back; (3) Videos: point at a local folder (or bucket/Drive later). Keep all three steps en/kn/hi-localized to stay consistent with the trilingual SPA work, and emit structured logs at each step per the project's launch-observability bar.

2. assistant (2026-06-21T13:56:13.681Z)

I'll verify each claim adversarially. Let me start by reading the relevant local files and pulling the repos to ensure I'm reading current state.

3. user (2026-06-21T13:56:19.322Z)

Already up to date.

---KHELSUTRA---

fatal: Cannot fast-forward to multiple branches.

4. user (2026-06-21T13:56:19.342Z)

```
540         print("=" * 60)
541
542
543     def _enforce_startup_or_exit(config) -> None:
544         """M5: run the per-profile startup checks for the server/CLI path and
545         ``sys.exit(1)`` on a
546         FATAL incoherent profile (e.g. cloud-serving pointed at local-only storage) BEFORE
547         any work.
548         A strict no-op when no ``deployment.profile`` is selected (today's behaviour). No
549         GPU probe
550         here ? the server stays torch-light; the detector healthcheck is the GPU authority.
551         Extracted
552         so the fail-fast wiring + the default-OFF no-op are unit-testable."""
553         try:
554             enforce_startup_checks(config, logger=logger)
555         except StartupCheckError as e:
556             logger.error("[STARTUP] refusing to start: %s", e)
557             sys.exit(1)
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
```

```

556     def main():
557         parser = argparse.ArgumentParser(description="Sports Highlight Indexer
Orchestrator")
558         parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
559         parser.add_argument("--output-dir", default="output", help="Directory where
processed clips/highlights are saved")
560         parser.add_argument("--setup", action="store_true", help="Launch dependency checker
and setup utility")
561         parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a
video clip to run detailed diagnostics on")
562         parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
563         parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
server on")
564         parser.add_argument("--max-frames", type=int, help="Limit processing to the first N
sub-sampled frames for debugging")
565
566         available_segmenters = list(segmenter_registry.list_available().keys())
567         parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
detection segmenter. Available: {available_segmenters}")
568         parser.add_argument("--trim-start", type=float, help="Start time in seconds for the
debug trim segment feature")
569         parser.add_argument("--trim-end", type=float, help="End time in seconds for the
debug trim segment feature")
570         parser.add_argument("--trim-output", help="Output filename for the debug trimmed
segment (saves in output-dir unless absolute path)")
571
572         args = parser.parse_args()
573
574         if args.setup:
575             # Invoke the diagnostics/bootstrap script (renamed from the old root
576             # setup.py to scripts/doctor.py so it no longer shadows the PEP 517
577             # build system). Resolve its path from this file so `indexer --setup`
578             # works regardless of the caller's cwd.
579             import subprocess
580             doctor_path = os.path.join(
581                 os.path.dirname(os.path.abspath(__file__)),
582                 "scripts", "doctor.py",
583             )
584             print(f"[INFO] Invoking diagnostics: {doctor_path} ...")
585             subprocess.run([sys.executable, doctor_path])
586             return
587
588         if args.trim_start is not None and args.trim_end is not None:
589             if not args.video_path:
590                 print("[ERROR] Please provide --video-path to extract a segment.")
591                 return
592
593             # Determine output path
594             out_filename = args.trim_output or "trimmed_segment.mp4"
595             if os.path.isabs(out_filename):
596                 out_path = out_filename
597                 out_dir = os.path.dirname(out_path)
598                 out_name = os.path.basename(out_path)
599             else:
600                 out_dir = args.output_dir
601                 out_path = os.path.join(out_dir, out_filename)
602                 out_name = out_filename
603
604             os.makedirs(out_dir, exist_ok=True)
605             print(f"[CLI] Extracting debug segment from {args.trim_start}s to
{args.trim_end}s from {args.video_path}...")
606
607             # global_config isn't initialized this early; load the typed config so the trim
608             # clip carries the same configured stitching paddings + watermark as a real

```

```

609         # highlight (watermark default-on).
610         stitching = load_app_config().stitching
611         compiler = VideoCompiler(out_dir, begin_padding=stitching.begin_padding,
612                                 end_padding=stitching.end_padding,
watermark=stitching.watermark)
613         try:
614             res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
615             print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
616         except Exception as e:
617             print(f"[ERROR] Failed to compile trimmed segment: {e}")
618         return
619
620     if args.debug_clip is not None:
621         if not args.video_path:
622             print("[ERROR] Please provide --video-path to debug a specific clip.")
623             return
624         run_cli_diagnose_clip(args.video_path, args.debug_clip)
625         return
626
627     # Initialize Global Config and DB
628     global global_config, db_instance
629     global_config = load_app_config() # returns typed AppConfig
630     _enforce_startup_or_exit(global_config) # M5: fail fast on an incoherent ACTIVE
profile
631
632     # The CLI --output-dir overrides the configured output directory. It now lives
633     # on the typed model (OutputConfig.output_dir), so every consumer ? including
634     # /api/compile ? reads the same value instead of a write-only runtime hack.
635     global_config.output.output_dir = args.output_dir
636     os.makedirs(global_config.output.output_dir, exist_ok=True)
637
638     db_path = os.path.join(global_config.output.output_dir,
global_config.output.db_name)
639     db_instance = Database(db_path)
640
641     # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
642     # the executor is in-process. Mark them failed so pollers see a terminal state.
643     swept = db_instance.fail_stale_jobs()
644     if swept:
645         logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
failed.")
646
647     # CLI processing mode (no web UI needed)
648     if args.video_path and not args.server:
649         print(f"[CLI] Processing video file: {args.video_path}")
650         # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is
fetched to scratch.
651         # The local-existence fast-fail only applies to a local input (stat the RESOLVED
key, not the
652         # raw URI); a remote fetch fails loudly. An unsupported scheme is a clean error,
not a crash.
653         storage_cfg = getattr(global_config, "storage", None)
654         try:
655             input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
656         except ValueError as e:
657             print(f"[FAIL] {e}")
658             return
659         if input_ref.backend == "local" and not os.path.exists(input_ref.key):

```

5. assistant (2026-06-21T13:56:22.635Z)

The khelsutra repo can't fast-forward. Let me check its state and proceed read-only, and continue verifying.

6. user (2026-06-21T13:56:23.891Z)

master

master...origin/master

?? .claude/

?? site/access.db

7. user (2026-06-21T13:56:25.304Z)

No matches found

8. assistant (2026-06-21T13:56:28.767Z)

Khelsutra is on master, up to date with origin/master (untracked files only). The grep for webbrowser/tray/--open in main.py returns nothing. Let me verify the health endpoint, the host binding, and read the cited khelsutra doc.

9. user (2026-06-21T13:56:30.174Z)

```
120         On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
121         fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
and
122         keep the prior good results intact ? a failed/empty re-run never destroys prior
data.
123         """
124         if segments:
125             db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
126         else:
127             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
128
129
130     # --- API Endpoints ---
131     @app.get("/api/config")
132     def get_config():
133         return global_config
134
135     @app.get("/api/health")
136     def health():
137         """Cheap liveness/readiness probe (no auth) for the SPA's offline banner
138         (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
139         confirm the engine is reachable and which detector is wired. Never raises."""
140         sport = getattr(global_config, "sport", None)
141         indexing = getattr(global_config, "indexing", None)
142         return {
143             "status": "ok",
144             "sport": sport,
145             "default_segmenter": getattr(indexing, "default_segmenter", None),
146         }
147
148     def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
149         """The full hash ? validate ? segment ? report pipeline for one video.
```

10. user (2026-06-21T13:56:30.600Z)

```
660         print(f"[FAIL] Video file not found: {args.video_path}")
661         return
662         local_video = storage.acquire_local_input(args.video_path, storage_cfg)
663
664         video_id = compute_video_id(local_video)
665         was_reingest = db_instance.get_video(video_id) is not None
666
667         # Validate (with-context variant surfaces ffmpegprobe_meta for the report)
668         print("[CLI] Validating...")
669         val_results, val_context = registry.run_all_with_context(local_video,
```

```

global_config)
670         failures = [r for r in val_results if not r.passed]
671
672         # Save video status
673         status = "failed" if failures else "processed"
674         db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r
in val_results])
675
676         seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
677         segmenter_actual = None
678         segmentation_ran = False
679         try:
680             print("\n" + "="*40)
681             print("VALIDATION SUMMARY")
682             print("="*40)
683             for r in val_results:
684                 status_symbol = "[PASS]" if r.passed else "[FAIL]"
685                 print(f"{status_symbol} {r.validator_name}: {r.message}")
686             print("="*40 + "\n")
687
688             if failures:
689                 print("[FAIL] Video failed checks. Indexing halted.")
690                 return
691
692             # Index
693             segmenter_cls = segmenter_registry.get(seg_name)
694             if not segmenter_cls:
695                 print(f"[FAIL] Segmenter '{seg_name}' not found.")
696                 return
697
698             print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
699             segmenter_actual = seg_name

```

11. assistant (2026-06-21T13:56:33.319Z)

`/api/health` confirmed: returns `{status, sport, default\_segmenter}` at line 142-146 (no auth).
Now let me find where the server actually starts (uvicorn / host binding).

12. user (2026-06-21T13:56:35.040Z)

```

7:import uvicorn
562:    parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
735:        uvicorn.run(app, host="127.0.0.1", port=args.port)

```

13. user (2026-06-21T13:56:41.034Z)

```

725         print(f"[REPORT] Machine JSON      : {paths.get('json')}")
726         return
727
728         # Start Local Server Mode
729         if args.server or not args.video_path:
730             print(f"[SERVER] Starting local FastAPI server on
http://localhost:{args.port}...")
731             # Resume self-scheduled work orphaned by the restart (EXECUTION_POLICY.md P3):
the
732             # in-process wake-timer for any parked 'waiting' job died with the old process,
so
733             # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
734             _get_executor().submit(_drain_due_jobs)
735             uvicorn.run(app, host="127.0.0.1", port=args.port)
736
737     if __name__ == "__main__":

```

```
738      # `python -m backend.main` runs THIS file as `__main__`, but `job_worker`/`uploads`  
resolve the  
739      # shared mutable state (`db_instance`, `global_config`) via `import backend.main` ?  
a SEPARATE
```

14. assistant (2026-06-21T13:56:44.390Z)

Confirmed: `uvicorn.run(app, host="127.0.0.1", port=args.port)` at line 735, with port defaulting to 8000. The host is hard-coded to `127.0.0.1` (not configurable), and port is configurable via `--port`. Now let me verify the console entry point and check the capabilities/presets engine seams.

15. user (2026-06-21T13:56:46.046Z)

```
1:# PEP 621 packaging for the badminton/sports highlight indexer (DEFERRED #7).  
17:name = "badminton-highlight-indexer"  
75:[project.scripts]  
76:# `indexer --server`, `indexer --help`, etc. -> backend/main.py::main()  
77:indexer = "backend.main:main"
```

16. user (2026-06-21T13:56:47.001Z)

No matches found

17. assistant (2026-06-21T13:56:51.133Z)

The console entry `indexer = "backend.main:main"` confirms claim 2's `indexer` console entry. But `/api/capabilities` returns NO matches in the backend. This is important for claim 13. Let me search more broadly.

18. user (2026-06-21T13:56:52.238Z)

Found 1 file
artifacts\appliance_design.json

19. user (2026-06-21T13:56:52.466Z)

backend\config\presets.py

20. assistant (2026-06-21T13:56:55.751Z)

Important finding: `/api/capabilities` does NOT exist anywhere in the engine code ? the only match for "capabilities" is in an `artifacts/appliance\_design.json` file. Let me check `presets.py` for PROFILE_PRESETS and search the API routes for any capabilities-like endpoint.

21. user (2026-06-21T13:56:57.802Z)

```
4:config knobs, so the truly-local / cloud-serving / cpu-mock modes don't drift apart one  
27:PROFILES: tuple[str, ...] = ("truly-local", "cloud-serving", "cpu-mock")  
33:    "truly-local": "local_gpu",  
34:    "cloud-serving": "cloud_run",  
43:PROFILE_PRESETS: dict[str, dict[str, Any]] = {  
44:    "truly-local": {  
45:        "deployment": {"profile": "truly-local", "compute_target": "local_gpu"},  
51:    "cloud-serving": {  
52:        "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},  
68:    return bool(profile) and profile in PROFILE_PRESETS  
74:    return copy.deepcopy(PROFILE_PRESETS.get(profile, {}))  
116:    - ``K_SERVICE`` set (Cloud Run) ? ``"cloud-serving"``  
118:    - ``probe_cuda`` and a CUDA GPU present ? ``"truly-local"``  
122:    imports torch; a CLI ``--detect`` or cloud-serving startup that wants the GPU probe  
passes  
127:    return "cloud-serving"
```



```
131:         return "truly-local"
```

22. user (2026-06-21T13:56:58.736Z)

```
backend\main.py:131:@app.get("/api/config")
backend\main.py:135:@app.get("/api/health")
backend\main.py:268:@app.post("/api/process", status_code=202)
backend\main.py:311:@app.get("/api/jobs/{job_id}/cost")
backend\main.py:326:@app.get("/api/videos/{video_id}/cost")
backend\main.py:337:@app.get("/api/jobs/{job_id}")
backend\main.py:370:@app.get("/api/highlights/{video_id}/{filename}")
backend\main.py:394:@app.post("/api/query")
backend\main.py:405:@app.post("/api/compile")
```

23. assistant (2026-06-21T13:57:03.856Z)

Key finding for claim 13: There is NO `/api/capabilities` endpoint in the engine. The API endpoints are: `/api/config`, `/api/health`, `/api/process`, `/api/jobs/.../cost`, `/api/videos/.../cost`, `/api/jobs/{job_id}`, `/api/highlights/...`, `/api/query`, `/api/compile`. `PROFILE_PRESETS` exists in `presets.py` with `truly-local`/`cloud-serving`/`cpu-mock`, but it is NOT surfaced via a `/api/capabilities` endpoint. The claim says compute placement "is already a setting surfaced from GET `/api/capabilities`" ? this endpoint does not exist. Let me confirm there's no other web framework registering it, and check the `auth.py` file that's new on this branch.

24. user (2026-06-21T13:57:05.274Z)

```
<tool_use_error>Path does not exist: C:/Users/avidu/Projects/badminton-highlight-indexer/backend/api/auth.py. Note: your current working directory is C:\Users\avidu\Projects\badminton-highlight-indexer.</tool_use_error>
```

25. user (2026-06-21T13:57:06.476Z)

```
1      # Single-box appliance ? UI-powered rally indexer + reel generator (KhelSutra Studio)
2
3      > **Status:** `DRAFT` ? owner-gated (auth choice, retention, cost ceiling, GPU host, P5
bridge) . **Owner:** Avi . **Created:** 2026-06-20 . **Last updated:** 2026-06-20
4      > **Lifecycle:** `DRAFT` ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when
DONE)
5      > **Tracking anchors:** §9 progress tracker is the source of truth; indexed in
`docs/README.md`; pointer in `NEXT_STEPS.md` + memory `current-state` once work starts.
6      > **Relation:** peer-of `PER_RALLY_SELECTION.md` / `REQUEST_ACCESS_FORM.md` in this
repo; **consumes** the engine's decoupled `StorageBackend` + `python -m backend.worker` seams
and the FastAPI loop; **extends** the engine's `HOSTING_PLAN.md` gating + `UI_PROPOSAL.md` alpha
screens. Engine-side changes here are **proposals to coordinate** with the active
decoupled/worker sessions ? not written unilaterally.
7      > **Honesty note:** every engine-fact claim is tagged `[verified]` (read against engine
source this session), `[design]` (proposed here), or `[researched]` (needs sign-off). Not
legal/financial advice.
8
9      ---
10
11     ## 0. TL;DR
12     **KhelSutra Studio** turns the engine from a CLI into a **single-box, UI-powered rally
indexer + reel generator**: the khelsutra `web/` React SPA, served behind an **edge auth gate**,
drives the **existing** engine FastAPI loop (ingest ? index ? pick rallies ? reel ? download) on
**one machine**, with `StorageBackend=local` for the MVP. It is the **single-box deployment
profile of the just-shipped decoupled architecture** ? storage=local + compute=local is the
degenerate collapse of the same storage/worker split.
13
14     The **single most important caveat:** the engine has **zero app-layer auth and CORS
`***` `[verified backend/main.py:42]`, so safety is structural **only** because the engine stays
bound to `127.0.0.1:8000` `[verified backend/main.py:638]`, a firewall blocks inbound `8000`,
and a reverse proxy/tunnel is the sole authenticated ingress ? **the gate, not the app, is the
```

```

tenancy boundary.**
15
16     Two honest hardware truths: (1) the current droplet is CPU-only ? real detection
there is Gemini (cloud) or stub (CPU mock); real local-GPU detection needs a GPU
host (native WASB, in-flight). (2) The owner priority is to stand up a GPU host ASAP ?
provider-agnostic (DO GPU droplet / GCE GPU VM / Cloud Run+GPU) ? but that runs in parallel
with the UI MVP, which ships on Gemini today.
17
18     ## 1. Problem & goal
19     The decoupled engine just shipped a `StorageBackend` ABC + a `python -m backend.worker
{infer|train}` dispatcher `[verified backend/storage/, backend/worker/__main__.py]`, and the
FastAPI loop already covers `process ? jobs ? query ? compile ? highlights` `[verified
backend/main.py]`. What's missing is a console that lets a non-CLI operator run the whole
loop on a single box, honestly, behind a gate.
20
21     Headline CUJ (owner's words): a hosted website where I pick an input ? a
Google Drive file, a bucket file (`s3://`/`gs://`), or a local upload ? it lands in the
bucket (DO Spaces / GCS) or locally ? I generate highlights on a GPU-enabled host
running the real e2e ? I pick rallies, watch, and download ? completely from the UI.
22
23     Outcome ("good"): an operator signs in, picks a source, watches it index, curates
rallies, downloads a reel ? on the live CPU droplet today (Gemini/stub), and on a GPU host
later (native WASB) with no UI fork.
24
25     Read to get current: `PER_RALLY_SELECTION.md` ($3/$4/$10 contract anchors), engine
`HOSTING_PLAN.md` (gating), `DECOUPLED_COMPUTE/HANDOFF.md` (what's built),
`VIDEO_LOCALITY_MODEL.md` (privacy/retention).
26
27     ## 2. Decisions locked
28     | # | Decision | Source / date | Implication |
29     |---|---|---|---|
30     | D1 | Reverse-proxied single origin; engine stays `127.0.0.1:8000` | `[verified
main.py:638]`; 2026-06-20 | CORS `` is moot from the browser ? the proxy is the boundary |
31     | D2 | Edge auth, zero engine auth code | `HOSTING_PLAN.md`; 2026-06-20 | Caddy
basicauth OR cloudflared + Cloudflare Access |
32     | D3 | MVP drives the in-process FastAPI worker loop, `storage=local` | `[verified
job_worker.py:37, local.py:33]`; 2026-06-20 | bucket worker = reserved scale path, out of MVP |
33     | D4 | Embed RallyPicker from `PER_RALLY_SELECTION.md` verbatim | PR [#11];
2026-06-20 | the console is a superset; do not redesign the picker |
34     | D5 | Console lives in the khelsutra web/ React scaffold | `web/README.md`;
2026-06-20 | Cloudflare-Pages-style root, not engine static |
35     | D6 | Compute placement is a setting, rendered from `GET /api/capabilities` |
`[design]`; 2026-06-20 | UI offers ONLY what the box can honestly do |
36     | D7 | Ingest is a source menu (local upload · bucket URI · Drive) normalizing to
one video object | `[verified backend/storage/ref.py parse_uri]`; 2026-06-20 | Drive is a
stub today ? scoped honestly ($4.2) |
37     | D8 | GPU host is provider-agnostic (DO droplet / GCE VM / Cloud Run+GPU); storage
stays the bucket | Owner 2026-06-20 | compute and storage are separate planes; no provider lock-
in |
38     | D9 | Observability is first-class ? structured logs on every new path + a DoD item
| Owner 2026-06-20 | see $5; verified during CUJ replay |
39     | D10 | Delivery = a self-hosted webserver + UI the user spins up (NOT a desktop
app); the user points at local storage + GPU/CPU OR cloud locations and the UI drives the
tool | Owner 2026-06-20 | supersedes `DESKTOP_APP_PLAN.md`; one artifact (this appliance)
runs on the owner box, a droplet, or a GPU host ? same UI, compute/storage are settings |
40
41     ## 3. Foundation ? what the engine gives us today `[verified this session unless
noted]`
42     - FastAPI loop: `POST /api/process` (202 + `job_id`) ? poll `GET /api/jobs/{id}` ?
`POST /api/query` ? `POST /api/compile` ? `GET /api/highlights/{video_id}/{filename}`
(FileResponse) `[main.py]`. Chunked upload exists ? `POST /api/upload/init` · `PUT
/api/upload/{id}/part/{i}` · `POST /api/upload/{id}/complete` · `DELETE /api/upload/{id}`
`[verified uploads.py:64,89,123,147]` ? but the bundled `app.js` never calls them (it only
hits `/api/process` + `/api/compile` `[verified app.js:88,373]`). So browser upload is client-
wiring only, no engine change.

```

```

43     - **In-process worker:** one module-global `ThreadPoolExecutor(max_workers=1)`
    `[verified job_worker.py:37]` ? strictly **one job at a time**; the DB `jobs` table is the
    clock; `fail_stale_jobs` sweeps queued/running on boot. Gemini chunk concurrency is a
    **separate** knob `indexing.ai_max_workers` (**default 5** `[verified models.py:183]`; set to 1
    to serialize) ? it is **not** already 1.
44     - **Storage:** `storage.backend` default = `"local"` `[verified models.py:298]`;
    `LocalStorage.fetch_to_local` is **identity** `[verified local.py:33]`, `put` no-ops on
    `src==dst` ? so on a single box the worker's pull?pipeline?push **collapses to read-local/write-
    local**. URI scheme `parse_uri` `[verified ref.py]`: bare/`local://` ? local; `s3://`
    (AWS/R2/**DO Spaces**/MinIO, by endpoint) ; `gcs://`/`gs://` ; `gdrive://` ? **follow-up stub
    (emulated only)**. Credentials never in config ? boto3 default chain / `~/aws` `[verified
    s3.py]`.
45     - **Compute seam:** `indexing.detector_impl` ? `{auto`=WSL, `native`=Linux-native,
    `stub`=CPU mock}; default `"auto"` `[verified models.py:129]`, resolved in
    `trajectory_hybrid._resolve_runner` `[verified :76-85]`. **Native WASB is in-flight on branch
    `feat/native-wasb-runner`** `native_wasb_runner.py` **exists** `[verified]`;
    **fusion_hybrid._build_native_runner` returns `NativeWasbRunner`** `[verified
    fusion_hybrid.py:86]` while **trajectory_hybrid._build_native_runner` still raises** `[verified
    :57-63]`. Acceptance gate = **owner byte-parity** (WSL CSV == native CSV) on a real GPU box ?
    not exercisable on the droplet or in the offline suite. Treat native as **not production-
    verified**.
46     - **Worker CLI:** `python -m backend.worker {infer|train}` pulls a `--video-uri` ?
    pipeline ? pushes `outputs/<video_id>/{intervals.csv,report.json}` `[verified
    worker/__main__.py:88,149]`. The validated GPU-free/key-free combo: `detector_impl=stub` +
    `--set indexing.skip_ai_handoff=true` + `storage.s3`.
47     - **Gating:** cloudflared Tunnel (outbound-only, no inbound ports) + Cloudflare Access
    email-OTP; backend trusts `Cf-Access-Authenticated-User-Email` `[HOSTING_PLAN.md]`. `GET
    /api/health` **now EXISTS** (engine M1, [#268]) ? `{status, sport, default_segmenter}`
    `[verified main.py:127, corrected 2026-06-21 ? was "does not exist"]`.
48     - **Droplet reality:** `168.144.126.176`, **CPU-only, NO GPU**, region `blr1`; co-hosts
    the marketing site (`/srv/khelsutra-site`, node `:8787`) **and** the engine (`~/rally`); ran the
    full train+infer e2e against DO Spaces with the **GPU mocked** `[SETUP_NEW_MACHINE.md,
    DECOUPLED_COMPUTE/HANDOFF.md]`. Shared testbed: MinIO `9000/9001` + GCS emulator `9023` also run
    there.
49     - **Absent (net-new):** `GET /api/capabilities`, `GET /api/videos` (only single-row
    `get_video(id)`), `GET /api/reels/{video_id}`, any source/proxy streaming endpoint, any
    **Dockerfile**, any per-video **cost ledger** `[verified ? grep returns none]`.

```

```

50
51     ## 4. Design / architecture
52
53     ### 4.1 Topology & request path
54     ONE box: **edge gate** (cloudflared+Access or Caddy basicauth) ? **reverse proxy /
    static host** serving the `web/` SPA bundle and proxying `/api/*` ? engine **FastAPI
    `127.0.0.1:8000`** ? **LocalStorage** ? **detector seam** (Gemini/stub today; native WASB on a
    GPU host). The firewall blocks inbound `8000` + the test ports `9000/9001/9023`. The marketing
    site stays a **separate** systemd unit on a **separate** subdomain.
55
56     ``
57     PUBLIC INTERNET
58         ? (1) EDGE GATE ? the ONLY ingress; auth lives HERE, not in the app
59         ?         cloudflared Tunnel + Cloudflare Access (no inbound ports) OR Caddy :443 +
    basicauth
60         ?
61         ?????????????????????????????????????????????????????????????????????????????????
62         ? ONE BOX   firewall BLOCKS inbound 8000 + 9000/9001/9023                                     ?
63         ? (2) reverse proxy / static host (Caddy/cloudflared)                                     ?
64         ?         /, /assets/* ? web/ React SPA           /api/*, /api/highlights/* ? :8000 ?
65         ?         ? ONE ORIGIN to the browser ? engine CORS '*' is irrelevant ?
66         ?         ?                                     ?
67         ? (3) ENGINE FastAPI (host=127.0.0.1:8000, HARD-bound [main.py:638]) ?
68         ?         control: jobs table + ThreadPoolExecutor(max_workers=1) drain ?
69         ?         data: uploads.py chunked upload · /api/compile · /api/highlights ?
70         ?         ?                                     ?
71         ? (4) STORAGE = LocalStorage (default) fetch=identity · put=no-op(src==dst) ?
72         ?         ?                                     ?

```

```

73      ? (5) COMPUTE seam: gemini (cloud, key) · stub (CPU mock) [droplet]          ?
74      ?                               native WASB (GPU host) · auto (owner Win+WSL)    ?
75      ?????????????????????????????????????????????????????????????????????
76      RESERVED SCALE PATH (out of MVP): console "compute target" selector ? storage=s3 (DO
Spaces)
77      ? SEPARATE GPU host runs `python -m backend.worker infer --video-uri s3://?` ?
outputs/<vid>/ back
78      (needs a NET-NEW orchestration endpoint reflecting outputs into the jobs/poll
contract)
79      ...
80
81      ### 4.2 Ingest sources (the headline) ? one menu, one normalized video object
82      All sources normalize to "a video the pipeline can read" ? a local path (MVP) or a
`StorageRef` (scale):
83
84      | Source | Path today | Status |
85      |---|---|---|
86      | **Local upload** | SPA chunks the file ? `max_part_mb` via the existing
`/api/upload/init\|part\|complete` `[verified uploads.py]` ? server path ? `POST /api/process`.
(Scale: also `StorageBackend.put` to `s3://`/`gcs://`.) | **wire client only** (endpoints exist)
|
87      | **Bucket URI** (`s3://`, `gs://`) | **POST /api/process` accepts the bucket URI
directly** (resolve ? fetch to scratch ? same pipeline ? API DB) `[verified main.py:159,274,283
? corrected 2026-06-21; was "net-new"]`; the worker CLI also fetches `--video-uri`. So the
**engine endpoint is NOT net-new** ? only the SPA call that POSTs the URI + polls `/api/jobs`
is. *(Default single-user mode; HOSTED mode w/ `allowed_video_root` correctly rejects a bucket
URI 400.)* | **engine: DONE (verified); SPA ingest call: design (= B-P2)** |
88      | **Google Drive** (`gdrive://`) | `parse_uri` recognizes it, but the gdrive backend is
an **emulated stub** `[verified ref.py, DECOUPLED_COMPUTE research]` | **stub ? roadmap;** MVP
requires a Drive?bucket pre-step; do **not** present Drive as live until the backend is real |
89
90      ### 4.3 Compute placement matrix (one UI, surfaced from `/api/capabilities`)
91      | Placement | Runs | Needs | Real/mock | Honest caveat |
92      |---|---|---|---|---|
93      | **CPU droplet ? gemini** (MVP target) | `segmenter=gemini`, in-proc worker |
`GEMINI_API_KEY`; network; **no GPU** | **REAL** | Cloud brain: uploads ?1280px chunks to Google
? **must be in consent copy**; **no per-video cost ceiling** today |
94      | **CPU droplet ? stub** | trajectory hybrid + `detector_impl=stub`+`skip_ai_handoff` |
nothing | **MOCK** | not real tracking ? UI must label "stub (CPU mock ? not real detection)";
demo/regression only |
95      | **CPU droplet ? motion** | `segmenter=motion` | nothing | REAL, low-quality | only no-
key/no-GPU real segmenter; never surface `server/receiver/events` (fabricated) |
96      | **GPU host ? native WASB** (in-flight) | `wasb`+`detector_impl=native` ?
`NativeWasbRunner` (in-proc torch) | a **GPU host ? DO GPU droplet / GCE GPU VM / Cloud
Run+GPU**; `WASB_WEIGHTS` | **REAL**, on-device | partially wired (fusion builds it, trajectory
refuses); **owner byte-parity gate**; **Cloud Run/GCE need a containerized worker ? no
Dockerfile exists, and WASB's torch-1.11 conda env makes the image non-trivial** |
97      | **Owner Win+WSL ? auto** | `wasb`+`detector_impl=auto` (WSL2+conda) | Windows+WSL2+GPU
| REAL (today) | Windows-only transport; the working real-GPU path for owner verification |
98      | **Remote GPU worker via bucket** (reserved) | console CPU + `storage=s3`; separate GPU
host runs `backend.worker infer` | bucket creds; a GPU host; a **net-new orchestration
endpoint** | REAL (remote) | FastAPI loop and worker CLI are **separate** today; "remote GPU as
a setting" is **design** ? selector shown **disabled** in MVP |
99
100     **Provider note:** **Cloud Run + GPU** (scale-to-zero, pay-per-job) is the strongest fit
for the engine's own **per-video-billing** economics (`DECOUPLED_COMPUTE` #246) ? gated on the
**containerization feasibility check** above.
101
102     ### 4.4 Auth / exposure (non-negotiable invariant)
103     No engine auth, by design; the gate is external. **Invariant (all three must hold):**
engine bound to `127.0.0.1` + firewall closes `8000` + edge gate up. Go-live checklist
`[HOSTING_PLAN.md:102-108]`: Access/basicauth ON; **rotate the chat-shared `GEMINI_API_KEY`**;
set `security.allowed_video_root`; rate-limit; per-user quota.
104
105     ### 4.5 UI operator console (web/ SPA ? superset of RallyPicker)

```

```

106      - **S1 Ingest & Process** ? the source menu (§4.2) + a segmenter picker from
`/api/capabilities` ? `POST /api/process`.
107      - **S2 Progress** ? 3s poll of the free-text stage
(`hashing/validating/segmenting/finalizing`), a **"one job at a time" notice, and the restart-
failure terminal state.
108      - **S3 Rally pick ? reel** ? **embed RallyPicker verbatim** (`PER_RALLY_SELECTION.md`):
honor `min_shot_count`; never render motion `server/receiver/events` or a fake confidence meter.
**Per-rally is ONE screen of the console.**
109      - **S4 Reels / History** ? needs `GET /api/videos` + `GET /api/reels/{video_id}` for
**reload-survival**.
110      - **S5 Settings** ? render `config` + `capabilities`; the compute/storage **target
selector shown DISABLED ("single-box: local") until the bucket-worker bridge lands.

```

26. assistant (2026-06-21T13:57:13.838Z)

Now I have critical context. The doc itself at line 90 says the matrix is "surfaced from `/api/capabilities`" but line 49 explicitly lists `GET /api/capabilities` under ****Absent (net-new)**** with `[verified ? grep returns none]`. And D6 (line 35) tags it `[design]`, not `[verified]`. So the claim that compute placement "is ALREADY a setting surfaced from GET /api/capabilities" is contradicted by the doc's own statement that the endpoint is absent/net-new and the design tag. Claim 13 is refuted.

Let me verify claim 1 (D10 exact text + supersedes), the cited line numbers. D10 is at line 39 ? but the claim cites line 39 (the source says line 39, claim said source :39). The claim cites `:39`. Let me verify the auth.py reference and D10 wording precisely. D10 text matches. Now let me verify claim 3's line citations (14, 30) and the backend/main.py:638 reference. The doc cites main.py:638 but the actual uvicorn line is 735. Let me check git history ? the doc may cite an older line.

27. user (2026-06-21T13:57:16.615Z)

```

=== blame main.py:735 (uvicorn) ===
c3dbd00 feat(gcp): always-install Ops Agent on GPU VMs + a cloud-GPU dry-run guide
=== Is there anything at main.py:638? ===
    db_path = os.path.join(global_config.output.output_dir, global_config.output.db_name)

```

28. user (2026-06-21T13:57:17.427Z)

```

110      - **S5 Settings** ? render `config` + `capabilities`; the compute/storage **target
selector shown DISABLED ("single-box: local") until the bucket-worker bridge lands.
111
112      ### 4.6 Reuse map
113      **Reused (no re-plumb):** the full `/api` loop; the in-process worker; the chunked
upload endpoints (un-called); LocalStorage default; the detector seam incl. the landing
`NativeWasbRunner`; the **RallyPicker** design + anchors; the `web/` scaffold; the
`HOSTING_PLAN` gate; the `SETUP_NEW_MACHINE` runbook. **Net-new:** Caddyfile + systemd units;
`GET /api/capabilities`; `GET /api/videos`; `GET /api/reels/{video_id}`; `GET /api/health`; the
SPA console screens; ingest-from-bucket wiring; a retention sweep; (reserved) the bucket-worker
bridge.
114
115      ## 5. Observability ? logging & monitoring `[required, first-class]`
116      Launch-readiness directive (owner 2026-06-20): **logs and monitoring taken seriously,
with a full log-analysis pass across every new code path.** The engine already uses a
centralized Python logger `[verified main.py:12-17]` ? carry that discipline into every new
path.
117
118      - **Correlation ID end-to-end.** Thread the existing `job_id` (from `/api/process`)
**plus** an `upload_id`/`request_id` through ingest ? process ? jobs ? compile ? highlights, and
into the worker for the bucket path, so **one CUJ is traceable** across UI ? FastAPI ? worker ?
GPU host. Log it on every line.
119      - **Structured logs.** New endpoints/screens emit structured (JSON-able) logs at **entry
/ result / error** ? not just "it ran". The SPA logs client-side errors + failed fetches with
the same correlation id.
120      - **Loud failures, never silent drops.** Carry the engine's ethos (#240 "surface failed

```

chunks loudly") + RallyPicker's `min\_shot\_count`-warning rule: a partial/empty reel must ****never**** look complete; every dropped clip/chunk is logged + surfaced in the UI.

121 - ****Health + metrics.**** Ship the missing `GET /api/health` `[HOSTING\_PLAN.md:42]` and a thin metrics surface (jobs queued/running/failed, last error, disk free, per-video cost once a ledger exists) for the engine-mode banner + alerting.

122 - ****Aggregation for transient hosts.**** A scale-to-zero Cloud Run / GPU host is ****ephemeral**** ? logs must ship to a durable sink (the bucket or a log service), not die with the instance.

123 - ****Log-analysis pass before merge (DoD item).**** Every PR that adds a code path includes a check that the path logs meaningfully, and the ****logs** are inspected during CUJ live-replay**` (\$8 Layers 1?3). Ties [[feedback-launch-quality-bar]].

124

125 **## 6. Risk table**

126 | Risk | Today | Mitigation |

127 |---|---|---|

128 | No app auth + CORS `` `[main.py:42]` | open if any of {localhost-bind, firewall, gate} slips | enforce all three; ****LIVE posture test**** that raw `:8000` is refused from outside |

129 | Test ports on shared droplet | MinIO `9000/9001` + GCS emu `9023` | firewall closes them |

130 | Retention | `output/`+`uploads/` originals ****never deleted**** | sweep / keep-proxy-only before any non-owner exposure |

131 | Surprise Gemini/GPU bill | ****no cost ceiling/ledger**** | per-video cap; `ai\_max\_workers` (default 5) tunable; `max\_workers=1` job serialization |

132 | Native WASB unverified | partially wired; owner-parity-gated | treat as not-production-verified until owner byte-parity on a real GPU box |

133 | New engine endpoints | `capabilities`/`videos`/`reels`/`health` don't exist | coordinate engine PRs with active sessions; ****branch from `origin/master`, re-verify HEAD, explicit `git add`**** |

134 | Drive ingest overstated | `gdrive://` is a stub | don't present Drive as live; require Drive?bucket pre-step until real |

135 | No Dockerfile | runbook-only reproducibility | feasibility-check a worker image before promising Cloud Run/GCE |

136 | Shared-checkout collisions | active worker sessions share the engine tree | the branch-safety protocol on every engine-side PR |

137 | ****Provisioning-credential leak**** | a DO API token / cloud creds could leak if pasted in chat or committed (cf. the ****already-rotated Spaces key**** exposed in chat, `DECOUPLED\_COMPUTE/HANDOFF.md`) | authenticate the CLI ****on-box**** (`doctl auth init` interactive / `gcloud` already authed) so the secret never enters chat or the repo; least-privilege token, revoke after; creds 0600, never committed; ****creating paid GPU infra is a confirm-first action every time**** |

138

139 **## 7. Honest limits ? what this does NOT do**

140 Does ****not**** add multi-tenant identity/RLS (single-tenant alpha; uploads/outputs global). Does ****not**** run real WASB on the CPU droplet (Gemini or stub only). Does ****not**** make the remote-GPU bucket path work (design; selector disabled). Does ****not**** provide in-UI source playback, job cancel, or a true progress bar (coarse free-text stage, poll-only). Does ****not**** containerize (runbook; no Dockerfile `[verified]`). Does ****not**** verify native WASB (owner byte-parity on a real GPU box, not this offline suite). A green SPA + contract tests prove the ****console plumbing****, not engine rally-detection quality (a separate track).

141

142 **## 8. CUJs & live-test plan** `[required]`

143 ****CUJs:**** ****CUJ-1**** ingest (Drive/bucket/local) ? index ? reel ? download (core); ****CUJ-2**** re-open a past match after reload; ****CUJ-3**** pick-by-query (deterministic filters + honesty-hiding); ****CUJ-4**** settings/compute placement (truthful capabilities).

144

145 ****Live-test plan ? 5 layers, every CUJ replayed LIVE throughout development:****

146 - ****L1 ? Mock engine (hermetic, fast, default + CI).**** A tiny FastAPI/MSW implementing the ****exact**** contract (upload; `process` 202; `jobs` state machine incl. restart-failure; `query` with a motion fixture for honesty-hiding; `config`; `capabilities` per box profile ? CPU-no-key / CPU-with-key / GPU; `compile`; `highlights` tiny mp4; the new `videos`/`reels`). The SPA drives ****all 4 CUJs in <1s****, ****replayed via the Claude Preview/browser tools**** (start dev server ? fill the ingest picker ? click through process/poll/RallyPicker ? assert the reel link via snapshot/network). Catches reload-survival (CUJ-2), the `play\_segments` field-name bug (CUJ-3), capability honesty (CUJ-4).

```

147 - **L2 ? Engine-contract checks (real FastAPI, GPU-free).** Boot `python -m backend.main
--server` with `storage=local` + `detector_impl=stub` + `skip_ai_handoff` and assert the **mock
and the real engine agree shape-for-shape** (202 keys, jobs enum, `result.play_segments` schema,
compile `{status,filepath}`, highlights content-type). Every new endpoint is born with a mock
handler **and** a real-engine contract test **in the same PR**. Runs in CI.
148 - **L3 ? Real local e2e (the MVP substrate).** Through the gate on the droplet: CUJ-1
with `stub` (key-free) and `gemini` (real, short clip to bound cost); the **exposure-safety
posture test** (raw `:8000/`:900x` refused from outside) as an assertion, not an assumption;
reuse the `parity.yml` dual-host pattern.
149 - **L4 ? CI regression (green gates merge).** `npm test` + a stub-engine contract job +
headless browser-replay CUJ scripts; one small PR per tracker row off `origin/master`, **no
stacking**; coverage at/above the floor; **logs inspected during replay** ($5).

```

29. assistant (2026-06-21T13:57:24.353Z)

This is very telling. The doc has multiple internal corroborations:

- Line 49 and line 113 and line 133: `GET /api/capabilities` is **net-new / does not exist**
`[verified ? grep returns none]`.
- D6 at line 35 is `[design]`.
- Line 14 cites `backend/main.py:638` for the 127.0.0.1 binding (stale line number ? the actual binding is at line 735 now, and line 638 is now a db_path line). The doc itself notes the engine ships PRs and line numbers drift.

So the binding fact (127.0.0.1:8000 hard-bound) is TRUE in current code (line 735), but the cited line number (638) is stale. The claim 3 substance is confirmed; the line citation is stale but that doesn't refute the substance.

Now let me verify the WASB / dual-torch packaging claims (claim 5) against the engine. Let me look for the torch version pins and conda env references.

30. user (2026-06-21T13:57:35.141Z)

```

8:# torch/torchvision are deliberately NOT listed in [project.dependencies]:
11:# `gpu` (documentation only) and the README. CI installs CPU torch on its own line.
26:# Runtime deps mirror requirements.txt, EXCEPT torch/torchvision (see header).
58:# `gpu` is DOCUMENTATION ONLY ? it is intentionally empty. torch/torchvision
62:# pip install torch>=2.12.0 torchvision>=0.27.0 --index-url
https://download.pytorch.org/whl/cu124
64:# pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
=== requirements files ===
requirements.txt
./backend/eval/calibrate_wasb.py
./backend/eval/export_wasb_segments.py
./output/19june_work/gx128_wasb.log
./output/wasb
./output/wasb/Adarsh v Avi__bb961d0d0cff_wasb_wasbcache
./output/wasb/GX010125_1080p__89605e80ed01_wasb.csv
./output/wasb/GX010125_1080p__89605e80ed01_wasb_wasbcache
./output/wasb/GX010128__db84f3e65318_wasb.csv
./output/wasb/GX010128__db84f3e65318_wasb_wasbcache
./output/wasb/GX010142_1080p__c4cfa94a792f_wasb.csv
=== search for torch pin in repo ===
./.claude/worktrees/agent-a771ff1d85e18f2ed/docs/archives/DEFERRED_HARDENING_PLAN-
COMPLETED-2026-06-14.md:33: - `gpu` ? torch/torchvision (note: the cu124 `--extra-index-url` is
**not**
./.claude/worktrees/agent-a771ff1d85e18f2ed/docs/archives/TEST_RUN_SUMMARY.md:53: pip install
torch torchvision --index-url https://download.pytorch.org/whl/cu124
./.claude/worktrees/agent-a771ff1d85e18f2ed/docs/archives/TEST_RUN_SUMMARY.md:55: (`setup.py`
auto-selects the matching CUDA wheel ? currently `cu124` ? when it detects an NVIDIA GPU.)
./.claude/worktrees/agent-a771ff1d85e18f2ed/docs/CODE_MAP.md:134:- `@scripts/doctor.py`
(formerly root `setup.py`) ? `::init_default_config` ? delegates to
`backend.config.save_default_config` (single-source defaults; skips if config.json exists).
`::run_pip_install` GPU-aware (`nvidia-smi` -> cu124 else cpu). `::run_diagnostics` (Python>3.8,
ffmpeg/ffprobe on PATH, optional torch+CUDA). Run via `python scripts/doctor.py` or `indexer
--setup`.

```

```

./..claude/worktrees/agent-a771ff1d85e18f2ed/docs/TRACKNET_WSL_SETUP.md:15:| WASB conda env |
`wasb` (Python 3.8, torch 1.11.0+cu113) ? verified `torch.cuda.is_available()==True` |
./..claude/worktrees/agent-a771ff1d85e18f2ed/pyproject.toml:62:# pip install torch>=2.12.0
torchvision>=0.27.0 --index-url https://download.pytorch.org/whl/cu124
./..claude/worktrees/agent-a771ff1d85e18f2ed/README.md:65:> pip install "torch>=2.12.0"
"torchvision>=0.27.0" --index-url https://download.pytorch.org/whl/cu124
./..claude/worktrees/agent-a771ff1d85e18f2ed/README.md:81:* Detect whether an NVIDIA GPU is
available (via `nvidia-smi`) and select the matching PyTorch wheel index automatically (`cu124`
if GPU present, `cpu` otherwise).
./..claude/worktrees/bhi-execpolicy/docs/archives/TEST_RUN_SUMMARY.md:53: pip install torch
torchvision --index-url https://download.pytorch.org/whl/cu124
./..claude/worktrees/bhi-execpolicy/docs/archives/TEST_RUN_SUMMARY.md:55: (`setup.py` auto-
selects the matching CUDA wheel ? currently `cu124` ? when it detects an NVIDIA GPU.)
./..claude/worktrees/bhi-execpolicy/docs/CODE_MAP.md:132:- `@setup.py` ? `::init_default_config`
? delegates to `backend.config.save_default_config` (single-source defaults; skips if
config.json exists). `::run_pip_install` GPU-aware (`nvidia-smi` -> cu124 else cpu).
`::run_diagnostics` (Python?3.8, ffmpeg/ffprobe on PATH, optional torch+CUDA).
./..claude/worktrees/bhi-execpolicy/docs/DEFERRED_HARDENING_PLAN.md:33: - `gpu` ?
torch/torchvision (note: the cu124 `--extra-index-url` is **not**)
./..claude/worktrees/bhi-execpolicy/docs/TRACKNET_WSL_SETUP.md:15:| WASB conda env | `wasb`
(Python 3.8, torch 1.11.0+cu113) ? verified `torch.cuda.is_available()==True` |
./..claude/worktrees/bhi-execpolicy/README.md:52:* Detect whether an NVIDIA GPU is available
(via `nvidia-smi`) and select the matching PyTorch wheel index automatically (`cu124` if GPU
present, `cpu` otherwise).
./..claude/worktrees/boxhill17/docs/archives/DEFERRED_HARDENING_PLAN-COMPLETED-2026-06-14.md:33:
- `gpu` ? torch/torchvision (note: the cu124 `--extra-index-url` is **not**)
./..claude/worktrees/boxhill17/docs/archives/TEST_RUN_SUMMARY.md:53: pip install torch
torchvision --index-url https://download.pytorch.org/whl/cu124
./..claude/worktrees/boxhill17/docs/archives/TEST_RUN_SUMMARY.md:55: (`setup.py` auto-selects
the matching CUDA wheel ? currently `cu124` ? when it detects an NVIDIA GPU.)
./..claude/worktrees/boxhill17/docs/CODE_MAP.md:136:- `@scripts/doctor.py` (formerly root
`setup.py`) ? `::init_default_config` ? delegates to `backend.config.save_default_config`
(single-source defaults; skips if config.json exists). `::run_pip_install` GPU-aware (`nvidia-
smi` -> cu124 else cpu). `::run_diagnostics` (Python?3.8, ffmpeg/ffprobe on PATH, optional
torch+CUDA). Run via `python scripts/doctor.py` or `indexer --setup`.
./..claude/worktrees/boxhill17/docs/TRACKNET_WSL_SETUP.md:15:| WASB conda env | `wasb` (Python
3.8, torch 1.11.0+cu113) ? verified `torch.cuda.is_available()==True` |
./..claude/worktrees/boxhill17/pyproject.toml:62:# pip install torch>=2.12.0
torchvision>=0.27.0 --index-url https://download.pytorch.org/whl/cu124

```

31. assistant (2026-06-21T13:57:40.914Z)

Important nuance for claim 5: The WASB conda env is documented as `torch 1.11.0+cu113` (TRACKNET_WSL_SETUP.md:15), confirming the separate-env fact. BUT the main engine's documented torch is `torch>=2.12.0 ... cu124` in the current pyproject.toml (line 62) and README ? NOT `torch 2.6+cu124` as the claim states. Let me verify the exact main-engine torch version in the canonical (non-worktree) files.

32. user (2026-06-21T13:57:44.489Z)

```

55         "obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3",
56     ]
57
58     # `gpu` is DOCUMENTATION ONLY ? it is intentionally empty. torch/torchvision
59     # CUDA wheels require `--index-url`/`--extra-index-url`, which PEP 621 cannot
60     # express, so they are NOT declarable here. Install them out-of-band instead:
61     # # CUDA 12.4 box:
62     # pip install torch>=2.12.0 torchvision>=0.27.0 --index-url
https://download.pytorch.org/whl/cu124
63     # # CPU-only box (also what CI uses):
64     # pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
65     gpu = []
66
67     # Storage backends (docs/DECOUPLED_COMPUTE/). The default `local` backend needs NONE of
68     # these. Cloud SDKs are imported lazily, so `import backend.storage` works without them;

```



```
69      # install the extra for the bucket you use. (boto3's S3 client covers AWS + Cloudflare
R2 +
```

33. user (2026-06-21T13:57:45.254Z)

```
|-----|-----|
| Miniconda | `~/miniconda3` |
| TrackNetV4 repo | `~/models/TrackNetV4` (https://github.com/TrackNetV4/TrackNetV4) |
| TrackNetV4 conda env | `TrackNetV4` (Python 3.9, TensorFlow 2.17 + bundled CUDA 12.3) |
| WASB repo | `~/models/WASB-SBDT` (https://github.com/nttcom/WASB-SBDT) |
| WASB conda env | `wasb` (Python 3.8, torch 1.11.0+cu113) ? verified
`torch.cuda.is_available()==True` |
| Staging dir for videos | `~/clips` (avoids slow `/mnt/c` per-frame reads) |
```

GPU: RTX 2070 Super visible via `nvidia-smi` inside WSL (CUDA-on-WSL working).

Activate an env with:

```
=== ffmpeg as system dep? ===
```

```
=== weights GCS bucket ===
```

```
backend/config/models.py:310: # input name is resolved against ``input_root`` (e.g.
``input_root``="gs://bucket/videos"`` + "x.mp4"
backend/config/models.py:311: # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is
empty, prefixed with ``input_backend``
backend/config/models.py:312: # (``"gs"`` + "bucket/x.mp4" => ``gs://bucket/x.mp4``). An
explicit scheme on the input
backend/config/models.py:313: # (``gs://?``/``s3://?``) or an absolute local path ALWAYS
wins. Non-local inputs are fetched to a
backend/config/models.py:326: # videos/ labels/ weights/ (e.g.
gs://bucket/workspaces/<video_id>/). "" = root directly.
```

34. assistant (2026-06-21T13:57:50.224Z)

Claim 5 substance confirmed with one correction: the main engine uses `torch>=2.12.0 ... cu124` (not "torch 2.6+cu124"). The WASB env is `torch 1.11.0+cu113` (confirmed). ffmpeg/ffprobe are NOT pip deps (system deps, confirmed ? doctor.py checks them on PATH). The "dual-torch + separate env + system ffmpeg + GCS weights ? single frozen binary can't represent the runtime" conclusion holds; only the specific main-engine torch version is slightly off (2.6 vs 2.12+). Let me verify the WASB weights GCS bucket fact more precisely.

35. user (2026-06-21T13:57:52.505Z)

```
config.json:61:      "weights_path": "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar",
pyproject.toml:84:[tool.hatch.build.targets.wheel]
deploy\cloudrun\README.md:15:- The WASB weights staged in the bucket (WASB-C3 unresolved ?
owner-gated): `gs://<bucket>/models/wasb_badminton_best.pth.tar`.
deploy\cloudrun\README.md:31:  --set-env-vars WASB_WEIGHTS=/staged/wasb_badminton_best.pth.tar
deploy\cloudrun\README.md:32:# (stage the weights into the container at start, or fetch from
gs:// in an init step ? WASB-C3.)
artifacts\landing\index.html:543:    if(e.target.name === 'segment') f.classList.toggle('is-
academy', e.target.value === 'academy');
deploy\digitalocean\gpu_setup.sh:52:WEIGHTS="$WASB_REPO/pretrained_weights/wasb_badminton_best.p
th.tar"
deploy\digitalocean\gpu_setup.sh:56: echo "[gpu]      !! MISSING $WEIGHTS ? fetch
wasb_badminton_best.pth.tar (Drive/bucket) before extraction."
deploy\gcp\README.md:8:> fall back to asia-southeast1 (Singapore) etc.; weights still land in
the asia-south1 bucket.
deploy\gcp\README.md:183:WASB weights (`wasb_badminton_best.pth.tar`) must be present at
deploy\gcp\README.md:184:~/models/WASB-SBDT/pretrained_weights/` (gpu_setup.sh does NOT fetch
them ? stage from Drive/bucket).
deploy\gcp\README.md:185:Point the native runner at them: `export WASB_WEIGHTS=~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar`
deploy\cloudrun\Dockerfile:16:# mount) and point $WASB_WEIGHTS at them. Resolve WASB-C3 before
any public, non-owner serving.
```

```
deploy\cloudrun\Dockerfile:56: WASB_WEIGHTS=/opt/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar \
backend\worker\_main_.py:435: # PUSH the versioned artifact bundle (weights.json +
manifest.json) to the bucket.
```

[Showing results with pagination = limit: 15]

36. assistant (2026-06-21T13:57:57.582Z)

Claim 5's "WASB weights live in a GCS bucket" is confirmed: `deploy/cloudrun/README.md:15` and `deploy/gcp/README.md:183-184` reference `gs://<bucket>/models/wasb\_badminton\_best.pth.tar`. Notably there IS a `deploy/cloudrun/Dockerfile` (claim 4's source mentions "no Dockerfile" but that's the appliance doc's older state; this doesn't affect claim 5's substance).

Now let me verify the external web claims (6-11) via search. These are about PyInstaller+CUDA size, Transformer Lab, Tauri, WebView2, pystray license, Inno Setup, Open WebUI. Let me batch the searches.

37. user (2026-06-21T13:58:06.451Z)

pystray Software License and Platform Support

****License:**** The repository uses LGPL-3.0 license, as indicated by the "COPYING.LGPL" file and license designation shown on the GitHub page.

****Supported Platforms:**** According to the README, pystray supports "_Linux_ under _Xorg_, _GNOME_ and _Ubuntu_, _macOS_ and _Windows_."

****Menu Items and Quit Action:**** The provided content does not contain specific information about whether pystray supports menu items or quit actions. The page only describes it as a library for creating "a _system tray icon_" without detailing feature capabilities. You would need to consult the full documentation at the referenced link to confirm these functionalities.

38. user (2026-06-21T13:58:09.116Z)

Web search results for query: "PyInstaller PyTorch CUDA bundle executable size 3-5GB cuDNN cannot split"

Links: [{"title": "PyInstaller, issue to build a package with CUDA, CuDNN and Tensorflow on Windows 10 · Issue #7175 · pyinstaller/pyinstaller", "url": "https://github.com/pyinstaller/pyinstaller/issues/7175"}, {"title": "Reducing the Size of PyTorch-Based Executable · pyinstaller · Discussion #8552", "url": "https://github.com/orgs/pyinstaller/discussions/8552"}, {"title": "Simplifying Deployment: Using PyInstaller with PyTorch ? codegenes.net", "url": "https://www.codegenes.net/blog/pyinstaller-pytorch/"}, {"title": "Pyinstaller with pytorch - PyTorch Forums", "url": "https://discuss.pytorch.org/t/pyinstaller-with-pytorch/47682"}, {"title": "Reducing the Size of PyTorch-Based Executable · Issue #8551 · pyinstaller/pyinstaller", "url": "https://github.com/pyinstaller/pyinstaller/issues/8551"}]

Based on the search results, I found relevant information about your query. Here's what the search revealed:

PyTorch with CUDA (torch2.3+cu118 in conda) is around 5GB, with the torch library in PyInstaller apps being about 4GB. This aligns with the 3-5GB executable size range you mentioned.

The key constraint you asked about is well-documented: Most of the size consists of binary extensions and CUDA/cuDNN shared libraries, which cannot be split. This explains why it's challenging to significantly reduce the bundle size when including CUDA support.

For optimization strategies, you can optimize package size by removing unnecessary imports and using the `--exclude-module` option to exclude certain modules that are not required. Additionally, if you are not using CUDA in your application, you can exclude CUDA-related modules, though this obviously doesn't apply if you need GPU acceleration.

The fundamental challenge is that the monolithic nature of CUDA/cuDNN libraries makes them difficult to decompose or split, which is why developers seeking smaller executables often have to choose between including full GPU support (resulting in large bundles) or sacrificing GPU capabilities altogether.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

39. assistant (2026-06-21T13:58:13.745Z)

Claim 6 confirmed (PyInstaller + PyTorch CUDA ? 3-5GB; CUDA/cuDNN shared libs cannot be split; small frozen torch apps are CPU-only/older-CUDA). Claim 10's pystray license = LGPL-3.0 confirmed, and platforms (Windows/macOS/Linux) confirmed. Let me verify the pystray menu/quit feature parity and check the remaining external claims: Transformer Lab, Tauri sidecar, WebView2, Inno Setup/winget, Open WebUI.

40. user (2026-06-21T13:58:21.670Z)

Transformer Lab Architecture & Packaging Approach

Architecture

Yes, Transformer Lab uses this design: "There is an Electron app built with React+Typescript. This app communicates with an API that handles machine learning tasks on top of our Python Engine."

PyInstaller Rejection

They rejected PyInstaller specifically because: "Transformer Lab allows users to install **plugins** that themselves can install arbitrary Python libraries. **So we needed an accessible Python environment that we could modify after it's installed on a user's machine.**"

Miniconda Implementation

They deployed an isolated, application-specific Miniconda installation with these defensive measures:

- Installation location: "we install our OWN conda in a Transformer Lab specific path, even if the user already has conda installed"
- Conda deactivation safeguards: "we deactivate conda three times (!) just to ensure we're not inside an existing conda"
- Dependency pinning: "we pin ALL package libraries using `pip-tools` to generate the requirements files"
- Environment isolation: "we then create a separate Transformer Lab environment and install a specific pinned version of Python to it"

They also manually handle problematic packages: "we **manually** install the flash_attention2 package because... fun... Flash Attention doesn't work with regular pip install alongside your other dependencies"

41. user (2026-06-21T13:58:22.249Z)

Web search results for query: "Tauri FastAPI sidecar backend silent failure port already in use Windows UTF-8 startup model load"

Links: [{"title": "Building Production-Ready Desktop LLM Apps: Tauri, FastAPI, and PyInstaller", "url": "https://aiechoes.substack.com/p/building-production-ready-desktop"}, {"title": "FastAPI Port Already in Use: Fix in 30 Seconds | Markaicode", "url": "https://markaicode.com/errors/fastapi-port-already-in-use-fix/"}, {"title": "GitHub - AlanSynn/vue-tauri-fastapi-sidecar-template: :package: Tauri v2 + Vue 3 (:zap: Vite) + FastAPI (:rocket:) | Desktop app template with Python sidecar backend. · GitHub", "url": "https://github.com/AlanSynn/vue-tauri-fastapi-sidecar-template"}, {"title": "[bug] pnpm tauri dev --no-watch error when starting dev server: · Issue #14284 · tauri-apps/tauri", "url": "https://github.com/tauri-apps/tauri/issues/14284"}, {"title": "How to use"}]

sidecar to run tauri with a console app? · tauri-apps/tauri · Discussion #5863", "url": "https://github.com/tauri-apps/tauri/discussions/5863"}, {"title": "[bug] [Windows / v2] beforeDevCommand is not killed when compiling failed with --no-watch · Issue #10023 · tauri-apps/tauri", "url": "https://github.com/tauri-apps/tauri/issues/10023"}, {"title": "Java sidecar for a Tauri + Angular app | by Jose I Santa Cruz G | ITNEXT", "url": "https://itnext.io/java-sidecar-for-a-tauri-angular-app-781a5d7d6db"}, {"title": "[bug] Error in tauri.conf.json crashes dev server · Issue #5173", "url": "https://github.com/tauri-apps/tauri/issues/5173"}]

Based on the search results, here's what I found about the specific issues you searched for:

Silent Failure with Port Already in Use

Port conflicts are the most common issue?if port 8000 is already in use, the backend fails to start silently.

****Windows Solution:****

Find the process with `netstat -ano | grep :8000` and kill it with `taskkill /PID <pid> /F`.

UTF-8 Encoding Issues on Windows

One mistake to avoid is UTF-8 encoding on Windows?Python's default console encoding causes crashes when llama.cpp outputs progress indicators with emojis or special characters.

****Fix in Tauri Rust Code:****

The Rust code spawns the sidecar during app startup with environment variables: ``.env("PYTHONUTF8", "1")`` and ``.env("PYTHONIOENCODING", "utf-8")``. These environment variables ensure UTF-8 encoding works correctly in the bundled Python environment.

Model Loading Issues

Model loading failures usually mean corrupted downloads?delete the models directory at `~/.tauri-llm-app/models/` and trigger a fresh download.

These are the most common issues when integrating a Tauri FastAPI sidecar backend on Windows, particularly related to startup failures and model loading.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

42. assistant (2026-06-21T13:58:27.409Z)

Claim 7 (Transformer Lab) is strongly confirmed ? Electron + Python engine, rejected PyInstaller because plugins install arbitrary Python libs needing a modifiable post-install env, ships its own conda in an app-specific path with pinned deps and defensive conda deactivation. Claim 8 (Tauri sidecar) confirmed: port-already-in-use silent backend failure is "the most common issue"; Windows UTF-8 crashes are a known gotcha. Let me verify the "5-10s startup with model load" specific and then check WebView2, Inno Setup, and Open WebUI.

43. user (2026-06-21T13:58:32.965Z)

Summary of Key Technical Points

**** (1) Backend Startup Time: ****

The article states: "Startup time is 5-10 seconds with model loading. Most of that is llama.cpp initializing the model in memory."

**** (2) Port 8000 Conflicts as Most Common Issue: ****

The article identifies this as the primary runtime problem: "Port conflicts are the most common issue. If port 8000 is already in use, the backend fails to start silently."

It provides diagnostic guidance: use ``netstat -ano | grep :8000`` to locate the conflicting

process and terminate it.

(3) Windows UTF-8 Encoding Crashes:

The article confirms encoding failures occur on Windows: "UTF-8 encoding on Windows. Python's default console encoding causes crashes when llama.cpp outputs progress indicators with emojis or special characters."

The solution involves setting environment variables before importing libraries to prevent these crashes from occurring in the bundled executable.

44. user (2026-06-21T13:58:35.597Z)

Web search results for query: "Tauri Windows WebView2 runtime preinstalled Windows 11 download installer fixed runtime bundle 180MB"

Links: [{"title": "Webview Versions | Tauri", "url": "https://v2.tauri.app/reference/webview-versions/"}, {"title": "Windows Installer - The Tauri Documentation WIP", "url": "https://jonaskruckenberg.github.io/tauri-docs-wip/building/windows-installer.html"}, {"title": "Windows Installer | Tauri", "url": "https://v2.tauri.app/distribute/windows-installer/"}, {"title": "Windows Installer | Tauri v1", "url": "https://v1.tauri.app/v1/guides/building/windows/"}, {"title": "Prerequisites | Tauri v1", "url": "https://v1.tauri.app/v1/guides/getting-started/prerequisites/"}, {"title": "Creating webviews doesn't account for fixed webview2 runtime in Windows environments without any runtime installed · Issue #13817 · tauri-apps/tauri", "url": "https://github.com/tauri-apps/tauri/issues/13817"}, {"title": "Setup for Windows | Tauri Studio", "url": "https://zhanglun.github.io/tauri-docs/docs/getting-started/setup-windows/"}, {"title": "Prerequisites - The Tauri Documentation WIP", "url": "https://jonaskruckenberg.github.io/tauri-docs-wip/prerequisites.html"}, {"title": "Need to include WebView2 runtime in compiled windows executable · tauri-apps/tauri · Discussion #4774", "url": "https://github.com/tauri-apps/tauri/discussions/4774"}, {"title": "Webview Versions | Tauri v1", "url": "https://v1.tauri.app/v1/references/webview-versions/"}]

Based on the search results, here's what I found regarding Tauri, Windows WebView2 runtime, and Windows 11:

WebView2 on Windows 11

WebView2 comes preinstalled on Windows 11, so users on that platform don't need to separately install the runtime.

Fixed Runtime Bundle and Installer Size

Distributing a fixed WebView2 Runtime version increases the Windows Installer by around 180MB. This matches the 180MB figure in your query.

How to Use Fixed Runtime

You can download the WebView2 fixed version runtime from Microsoft's website, and then run `tauri build` to produce the Windows Installer with the fixed WebView2 runtime.

Default Installer Behavior

By default, the Tauri installer downloads and installs the Webview2 Runtime if it is not already installed (On Windows 11, WebView2 is preinstalled).

The search results provide official Tauri documentation on how to configure WebView2 distribution with your Windows installers, including the option to bundle a fixed runtime version for offline installations or specific version control needs.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

45. assistant (2026-06-21T13:58:40.390Z)

Claim 8 fully confirmed (5-10s startup with model load; port 8000 silent failure most common; Windows UTF-8 crash). Claim 9 (WebView2) confirmed: preinstalled on Windows 11, Tauri installer downloads/installs if absent, fixed runtime bundle ? +180MB. The claim's "and recent Win10" needs a quick check ? WebView2 is preinstalled on Win11 but on Win10 it's via Edge/Evergreen rollout. Let me verify the Win10 nuance and then Inno Setup + winget + Open WebUI.

46. user (2026-06-21T13:58:47.529Z)

WebView2 Runtime on Windows

Preinstallation Status

According to the documentation: "On Windows 10 (April 2018 release or later) and Windows 11, the WebView2 runtime is distributed as part of the operating system."

This means both Windows 10 and Windows 11 typically have WebView2 included by default, eliminating the need for separate installation on these versions.

Installer WebView2 Options

Tauri provides four primary approaches for handling WebView2 installation:

1. **Downloaded Bootstrapper** (default): Fetches and executes the bootstrapper during installation. Minimal installer overhead but requires internet connectivity. Not ideal for Windows 7 via `.msi` files.
2. **Embedded Bootstrapper**: Bundles the bootstrapper within the installer, adding approximately 1.8MB. Requires internet but offers better Windows 7 compatibility.
3. **Offline Installer**: Embeds the complete WebView2 installer, increasing package size by roughly 127MB. Enables offline installation scenarios.
4. **Fixed Version**: Distributes a specific WebView2 runtime version, expanding the installer by approximately 180MB. Grants direct control over runtime patching and distribution.

Each option balances installer size, connectivity requirements, and deployment flexibility based on your application's target environment.

47. user (2026-06-21T13:58:47.573Z)

Inno Setup: Key Questions Answered

Is it free/open source?

Yes. The page describes Inno Setup as "an open-source installation builder" and notes that "Full source code is available from GitHub."

Single self-contained EXE support?

Yes. The documentation mentions it "Supports creation of a single EXE to install your program for easy online distribution."

Non-administrative installations?

Yes. Inno Setup provides "Extensive support for both administrative and non administrative installations."

Pascal scripting support?

Yes. It includes an "Integrated Pascal scripting engine option for advanced run-time install and uninstall customization."

License?

Inno Setup is copyrighted software with restrictions on distribution and use detailed in the

LICENSE.TXT file. The page specifically requests that "All commercial users of Inno Setup are requested to purchase a commercial license," indicating it requires payment for commercial use despite being open source.

48. assistant (2026-06-21T13:58:53.291Z)

Important nuance for claim 9: The official Tauri docs say WebView2 "is distributed as part of the operating system" on **Windows 10 (April 2018 release or later) AND Windows 11**. So the claim's framing "preinstalled on Windows 11 (and recent Win10), but on OLDER Windows the installer must download/install it" is essentially correct ? older = pre-April-2018 Win10 and Win7. Confirmed.

Claim 11 (Inno Setup): "open-source installer builder" confirmed. Single-EXE distribution confirmed. Non-admin installs confirmed. Pascal scripting confirmed. The claim says "open-source" ? accurate (source on GitHub, free to use including commercially, though a commercial license is *requested* not required). Now let me verify winget ingests INNO/EXE/MSI installer types, and the Open WebUI first-run flow.

49. user (2026-06-21T13:58:59.846Z)

Web search results for query: "winget manifest InstallerType supported inno exe msi nullsoft wix burn"

Links: [{"title": "--installer-type exe` filter should work with `nullsoft`, `inno` and `burn` · Issue #4702 · microsoft/winget-cli", "url": "https://github.com/microsoft/winget-cli/issues/4702"}, {"title": "Packages - Windows Package Manager (WinGet)", "url": "https://mintlify.wiki/microsoft/winget-cli/concepts/packages"}, {"title": "How does winget choose an MSI over an EXI when the manifest has both? · microsoft/winget-cli · Discussion #3497", "url": "https://github.com/microsoft/winget-cli/discussions/3497"}, {"title": "`winget install` support specifying installer extension/type (EXE vs. MSI) · Issue #2146 · microsoft/winget-cli", "url": "https://github.com/microsoft/winget-cli/issues/2146"}, {"title": "Added more info for \"Installer Types\" values in Settings.md. (#6067) · microsoft/winget-cli@c8fe1ea", "url": "https://github.com/microsoft/winget-cli/commit/c8fe1ea01ee7cc92923cdff260359692d4008862"}, {"title": "winget-pkgs/doc/manifest/schema/1.6.0/installer.md at master · microsoft/winget-pkgs", "url": "https://github.com/microsoft/winget-pkgs/blob/master/doc/manifest/schema/1.6.0/installer.md"}, {"title": "Release Windows Package Manager 1.29.240 · microsoft/winget-cli", "url": "https://github.com/microsoft/winget-cli/releases/tag/v1.29.240"}, {"title": "settings command (winget)", "url": "https://learn.microsoft.com/en-us/windows/package-manager/winget/settings"}, {"title": "winget-pkgs/README.md at 7115b91ea165bfb935b8866b9f8a18114d7b5d · microsoft/winget-pkgs", "url": "https://github.com/microsoft/winget-pkgs/blob/7115b91ea165bfb935b8866b9f8a18114d7b5d/README.md"}, {"title": "InstallerType: Wix vs Burn? · Issue #261 · microsoft/winget-cli", "url": "https://github.com/microsoft/winget-cli/issues/261"}]

Based on the search results, here's information about Windows Package Manager (winget) manifest InstallerType supported values:

Supported InstallerType Values

The supported InstallerType values in winget manifests include: "msix", "msi", "appx", "exe", "zip", "inno", "nullsoft", "wix", "burn", "pwa", "portable", and "font".

More specifically, allowed values as of version 1.12.470 include: appx, burn, exe, font, inno, msi, msix, msstore, nullsoft, portable, wix, and zip.

Installer Types Overview

The Windows Package Manager supports MSIX, MSI, and executable installers. Some well known formats (Inno, Nullsoft, WiX, and Burn) provide standard sets of installer switches to provide different installer experiences.

Default Behavior

By default, and with all other properties being equal, WinGet defaults to the installer type that is listed first in the manifest's installer YAML if the package has not been installed yet. The matching parameter is `--installer-type`, which will override the settings.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

50. user (2026-06-21T13:59:00.007Z)

Open WebUI First-Run Flow

Based on the documentation provided, the getting-started process involves:

1. ****Access the interface****: Open localhost in your browser
2. ****Configure a provider****: Navigate to "Admin Settings ? Connections" and enter a URL and API key
3. ****Models appear automatically****: "Open WebUI auto-detects available models from most providers"

The documentation states that the basic workflow is:

- "You type a message in Open WebUI"
- "Open WebUI sends it to your provider's API endpoint"
- "The provider runs inference on the selected model"
- "Tokens stream back to Open WebUI in real time"

Regarding model management, the page does not explicitly describe downloading or pulling models from within the app itself. Instead, it emphasizes that "Adding a provider is as simple as entering a URL and API key" and models are auto-detected from your chosen provider (whether Ollama, OpenAI, local servers, etc.). The actual model management appears to happen through your selected provider, not directly within Open WebUI's interface.

51. assistant (2026-06-21T13:59:07.004Z)

Claim 11 (winget): supported InstallerType values include ``inno``, ``exe``, ``msi`` ? confirmed.
Claim 12 (Open WebUI precedent): open localhost in browser ? enter provider URL + API key in settings (Admin Settings ? Connections) ? models auto-detected. The "create an account" and "pull models from an in-app browser" details aren't fully on this specific page (account creation is a separate Open WebUI first-login step that does exist; in-app model pull is an Ollama feature). The claim is tagged medium confidence and the core precedent (in-UI first-run wizard for provider URL + API key) is confirmed. The "pull models from an in-app browser" is more an Ollama integration detail than something this page substantiates ? partially supported.

Now let me confirm pystray's menu/quit feature parity claim, and verify the `DESKTOP_APP_PLAN.md` / native-runner references in claim 4/5. Let me also double-check D10 supersedes wording and the `appliance_design.json` artifact to understand "capabilities."

52. user (2026-06-21T13:59:11.973Z)

=== grep supersede in appliance doc ===

```
39:| D10 | Delivery = a **self-hosted webserver + UI the user spins up** (NOT a desktop app);
the user points at **local storage + GPU/CPU OR cloud locations** and the UI drives the tool |
Owner 2026-06-20 | **supersedes `DESKTOP_APP_PLAN.md`**; one artifact (this appliance) runs on
the owner box, a droplet, or a GPU host ? same UI, compute/storage are settings |
190:**Engine code `[verified this session]`**
`backend/main.py:12-17,42,52,307-328,331-340,342-396,638`;
`backend/api/uploads.py:64,89,123,147`; `backend/api/job_worker.py:37`;
`backend/api/static/app.js:88,344,373`;
`backend/storage/{base.py, local.py:33, ref.py, __init__.py:43}`;
`backend/config/models.py:129,183,298`;
`backend/pipeline/segmenters/{trajectory_hybrid.py:57-63,76-85, fusion_hybrid.py:76-86,
```



```
gemini.py:470-485, motion.py:191-198}`; `backend/pipeline/detectors/{native_wasb_runner.py, stub_runner.py}`; `backend/worker/___main__.py:88,98,149`; `config.json` (`default_segmenter: gemini`). **Engine docs:** `HOSTING_PLAN.md:42,102-108`; `VIDEO_LOCALITY_MODEL.md`; `DESKTOP_APP_PLAN.md`; `SETUP_NEW_MACHINE.md`; `PLATFORM_ARCHITECTURE.md`; `DECOUPLED_COMPUTE/{README,02,04,07,HANDOFF}.md`; `UI_PROPOSAL.md:41-66`; `PROJECT_DOC_TEMPLATE.md`. **This repo:** `PER_RALLY_SELECTION.md`, `REQUEST_ACCESS_FORM.md`, `PORTABILITY.md`, `web/README.md`.
```

53. user (2026-06-21T13:59:14.309Z)

pystray Menu Support Across Platforms

Menu and Menu Items: Yes, pystray supports creating menus with menu items. You can attach a `pystray.Menu` instance containing `MenuItem` objects to an icon via the `menu` keyword argument.

Quit Functionality: The documentation shows how to create menu item callbacks that can call `icon.stop()` to halt execution, though no explicit Quit example is provided in this content.

Cross-Platform API: pystray aims for "a unified API for all supported platforms," with the same menu and icon API working across Windows, macOS, and Linux.

Platform Limitations:

- **Linux (Xorg):** "Menus are not supported on X," meaning the xorg backend offers only "a default action" and no broader menu functionality.
- **Linux (AppIndicator):** All features work except "a menu default action" is unsupported.
- **macOS:** Radio button menu items are unsupported; check `Icon.HAS_MENU_RADIO` at runtime.
- **macOS/AppIndicator:** Default menu items are unsupported; check `Icon.HAS_DEFAULT` at runtime.
- **General Linux limitation:** "Menus also support a default item...On X, this item will be activated when the user clicks on the icon using the primary button."

Windows and macOS have the most complete menu support, while Linux backends have varying limitations depending on which backend is available.

54. assistant (2026-06-21T13:59:22.707Z)

Important refinement for claim 10: pystray does NOT have "full feature parity" for menus across all three platforms. Windows/macOS have full menu support, but on **Linux/Xorg** "Menus are not supported" (only a default action), and AppIndicator/macOS have specific limitations (default-action / radio-button). So the claim "supports Windows/macOS/Linux with full feature parity for icon + menu + Quit" is overstated? there are documented Linux menu limitations. However, the launcher's target is Windows (Inno Setup installer), where pystray menu + Quit work fully, so the practical conclusion (use pystray for a Windows tray) holds, but the literal "full feature parity" across all platforms is refuted. The LGPL-3.0 license fact and guardrail conflict (project requires MIT/Apache/BSD) is fully confirmed.

D10 supersede wording is confirmed verbatim at line 39 of LOCAL_APPLIANCE_ARCHITECTURE.md (note: the claim's source cited `:39` matches). And `DESKTOP_APP_PLAN.md` is an engine doc that exists (referenced in the badminton repo). Let me quickly confirm `DESKTOP_APP_PLAN.md` exists in the engine repo for claim 4/5 references.

55. user (2026-06-21T13:59:25.317Z)

docs/DESKTOP_APP_PLAN.md

=== status line + de-WSL native runner mention ===

```
3:> **Status:** `DRAFT` ? `owner-gated` (scoping doc only; **nothing is built**) . **Owner:**  
avidullu . **Created:** 2026-06-18 . **Last updated:** 2026-06-18  
29:**Read to get current:** `VIDEO_LOCALITY_MODEL.md` (the L0/locality framing + Q05 packaging  
seed), `PLATFORM_ARCHITECTURE.md` §3.2 (the ComputeBackend seam),  
[`backend/pipeline/detectors/base.py`](../backend/pipeline/detectors/base.py) (the detector ABC  
that already anticipates a Linux-native runner), `tools/generate_highlights.py` (the local  
entrypoint this app wraps).
```

```

39:| D5 | **Reuse the existing pipeline verbatim; add only the shell + a native runner + device-
IO.** | `backend/pipeline/detectors/base.py` reuse contract `[verified]` | No re-architecture of
windowing/stitch. New code = native runner, USB/SD detection, app shell, packaging. |
50:### 3.2 The de-WSL native runner is already designed-for `[verified]`
51:The detector layer was deliberately built to make this port low-regret.
[ `backend/pipeline/detectors/base.py` ](../backend/pipeline/detectors/base.py) documents,
verbatim, an Optional Linux-native path (the main de-risk goal): a
`LinuxNativeRunner`/`ONNXRunner` that "Skip[s] all wsl / _stage / conda activate, load[s]
weights locally (torch or onnx)? write[s] the exact same CSV format so
`trajectory_to_action_windows` works verbatim." The WSL plumbing lives in a separate
`WslCommandMixin`, explicitly so a native runner "must not inherit WSL plumbing." The actual
inference math is already isolated in
[ `wasb_infer.py` ](../backend/pipeline/detectors/wasb_infer.py) (torch + the WASB-SBDT modules,
emitting `Frame,Visibility,X,Y`). **So the port is: lift `wasb_infer.py` out of WSL, load
weights with native torch/onnxruntime per-OS, keep the identical CSV contract.** The windowing
brain (`trajectory_to_action_windows`) and stitch are untouched.
63:- **Backend packaging: the Python/FastAPI backend bundles via **PyInstaller** or a frozen
venv as a local "sidecar." Packaging was already modernized ? **PEP 621 `pyproject.toml` + an
`indexer` console entry point** shipped (DEFERRED_HARDENING_PLAN` #7, DONE, PR #167)
`[verified]` ? so a CLI MVP has a real head start.
96:| **Native (de-WSL) WASB runner, per-OS** | **NEW** `[design]` | `NativeWasbRunner` ? sibling
of `wasb_runner.py`, no `WslCommandMixin` |
99:| **Packaging + code-signing per OS** | **NEW** `[design]` | PyInstaller/MSIX (Win),
.dmg+notarize (mac), AppImage/Flatpak (Linux) ? P3 |
105:Lean: **start with a CLI + installer MVP** (wraps the existing
`indexer`/`generate_highlights.py` entry point ? fastest way to validate packaging, signing,
USB-detect, and the native runner end-to-end on all three OSes), then **graduate to a Tauri
GUI** for the non-technical user (tiny download fits the "huge-video laptops, slim app" frame;
the Python backend runs as a sidecar the Tauri front-end calls over localhost). Electron is the
fallback if webview inconsistencies bite. This mirrors the repo's "simplest thing first,
graduate on evidence" bias.
131:| **P1** | **De-WSL native detector port** (`NativeWasbRunner`, per-OS): CUDA-native
(Win/Linux), MPS/CPU fallback, identical `Frame,Visibility,X,Y` CSV; reuse windowing verbatim;
healthcheck per backend. | P0 verdict | Yes | ? | ? |
150:- [ ] **P1** the native (de-WSL) detector runs on ?1 GPU OS and ?1 CPU/MPS path, producing
byte-compatible trajectories (regression vs the WSL runner on a shared clip).
164:- `2026-06-18` ? Created. Scoping doc for the cross-platform desktop app + the de-WSL
native-compute port; all `[design]`. $7 P0 (compute-feasibility spike) named the make-or-break
gate. Owner-gated; nothing built.

```

56. assistant (2026-06-21T13:59:31.903Z)

This confirms claim 4's DESKTOP_APP_PLAN.md:37 region (the de-WSL native runner is at §3.2, line 50-51, designed but `[design]`/not built; "nothing is built" per status line 3). Note the cited line for claim 4 is `:37` (D5 is at line 39 region ? the de-WSL native runner discussion is around there). The substance holds.

I now have everything I need. Let me also note one cross-check: DESKTOP_APP_PLAN.md itself (line 105) actually proposes "graduate to a Tauri GUI" with the Python backend as a sidecar ? and explicitly at line 63 mentions PyInstaller/frozen-venv for backend packaging. This is relevant to the recommendation's argument that option (c) "presumes a PyInstaller freeze" ? the engine's own DESKTOP_APP_PLAN does pair Tauri-sidecar with PyInstaller, supporting that linkage as the planned (now-superseded) path.

I have verified all 13 claims. Let me compile the structured output.

57. user (2026-06-21T14:00:44.801Z)

Structured output provided successfully

58. assistant (2026-06-21T14:00:52.996Z)

Verification complete. I checked all 13 claims adversarially against current code (both repos confirmed fresh against origin) and the cited web sources.

Headline results:

- 10 confirmed, 2 confirmed-with-correction, 1 refuted.
- The one refutation is the load-bearing one: claim 13's `GET /api/capabilities` endpoint does not exist in the engine ? the khelsutra appliance doc itself lists it as net-new/[design]` (lines 35/49/113), and grep finds zero hits. The underlying `PROFILE_PRESETS` (truly-local/cloud-serving/cpu-mock) do exist in `backend/config/presets.py`, so the config is real but the HTTP surface the wizard step relies on is unbuilt.
- Two corrections that don't change the recommendation: main-engine torch is `>=2.12.0+cu124` (not 2.6); pystray's "full cross-platform menu parity" is overstated (Linux/Xorg has no menus) ? though the LGPL-3.0 guardrail conflict, which is the load-bearing point, is fully correct.
- Several cited engine line numbers are stale-but-true (binding moved from main.py:638 to :735; health from :127 to :135) ? substance intact.

The recommendation's core thesis holds; full per-claim detail is in the StructuredOutput.